Factory Performance & Loss Intelligence Platform (FPLIP)

A Contract-Governed, Sandbox-Constrained Manufacturing Data Lakehouse

with AI-Grounded Loss Diagnostics

Lily
École Nationale des Sciences Appliquées (ENSA) Safi
AI & Data Science – IDIA Specialization
End-of-Studies Internship Report

August 2026

Acknowledgments

I express my sincere gratitude to my academic supervisors at École Nationale des Sciences Appliquées (ENSA) Safi for their guidance, critical reviews, and constant support throughout this internship. I also extend my gratitude to the host company management and technical engineers for providing the industrial context, operational data specifications, and domain feedback that motivated this project.

Abstract

Manufacturing lines producing concrete blocks, pavement blocks, and zellige tiles routinely lose productive capacity to unplanned downtime, speed degradation, and scrap material waste. Without a governed analytical data platform, operational losses remain invisible until reflected as missed production output. This report presents the design, implementation, and empirical validation of the Factory Performance & Loss Intelligence Platform (FPLIP), a zero-cost, contract-governed data lakehouse engineered over a nine-week internship to eliminate this visibility gap. FPLIP ingests raw production, downtime, and quality-control logs into a Bronze/Silver/Gold medallion architecture hosted on Google BigQuery Sandbox. The platform calculates Overall Equipment Effectiveness (OEE) and scrap root-cause metrics via dbt transformations, serving data through a cached FastAPI tier, an Apache Superset dashboard, and a role-based Laravel investigation application augmented with a machine-learning scrap-risk model and a tool-calling AI diagnostic assistant. A central technical contribution of this work is the demonstration of architectural discipline under platform constraints: BigQuery Sandbox restricts Data Manipulation Language (DML) statements, which required implementing idempotent partition-decorator loads, full-refresh dbt materializations, and a hand-rolled, DDL-only Slowly Changing Dimension Type 2 (SCD2) history mechanism in place of standard `dbt snapshot`/`MERGE` patterns. All major technical decisions are documented across eighteen Architectural Decision Records (ADRs). System robustness was validated through a cold-restart test that rebuilt the entire platform from an empty GCP project in under eighteen minutes, alongside a synthetic data quality audit that surfaced and resolved foreign-key integrity gaps and a hardcoded credential in the machine-learning endpoint prior to deployment. This report presents the platform's architectural design, implementation methodology, and performance evaluations, positioning FPLIP as a reproducible template for building agent-safe, contract-governed data lakehouses under realistic infrastructure limits.

Contents

Acknowledgments **1**

Abstract **2**

List of Abbreviations **8**

1 Introduction **10**
1.1 Industrial Context . . . 10
1.2 Problem Statement . . . 10
1.3 Project Objectives . . . 11
1.4 Research Contributions . . . 11
1.5 Scope and Boundary Constraints . . . 12
1.6 System Requirements . . . 12
1.7 Project Management . . . 12
1.8 System Limitations . . . 13
1.9 Report Roadmap . . . 13

2 Literature Review **14**
2.1 Medallion Architecture in Modern Data Lakehouses . . . 14
2.2 Data Contracts and Shift-Left Governance . . . 14
2.3 Idempotency and Load Safety in Data Pipelines . . . 14
2.4 Slowly Changing Dimensions (SCD Type 2) . . . 15
2.5 DML vs. DDL Constraints in Cloud Sandbox Environments . . . 15
2.6 OEE as an Analytical Framework . . . 15
2.7 Data Engineering Testing Taxonomies . . . 16
2.8 Grounding and Tool-Calling in AI Applications . . . 16
2.9 Orchestration and CI/CD for Ephemeral Infrastructure . . . 16
2.10 Comparison with Existing Solutions . . . 17

3 System Architecture **18**
3.1 High-Level Platform Architecture . . . 18
3.2 Technology Choices and Rationale . . . 18
3.3 GCP Resource Topology and IAM Security Design . . . 19
3.4 Engineering Methodology . . . 19
3.5 Comparison with Alternatives . . . 20
3.6 Orchestration Overview . . . 20
3.7 CI/CD Pipeline & Ephemeral Isolation . . . 21
3.8 Security Architecture . . . 21
3.9 Risk Analysis & Mitigation . . . 21

3.9.1 Disaster Recovery & Business Continuity . . . 22

4 Medallion Data Pipeline **23**
4.1 Bronze Layer . . . 23
4.2 Silver Layer . . . 24
4.3 Gold Layer . . . 25
4.4 Data Quality Management & Validation . . . 26
4.4.1 Detailed Data Quality Metrics . . . 27
4.5 Performance Tuning & Scalability . . . 28
4.5.1 Data Volume & Scalability Projections . . . 28
4.6 Testing Strategy & Quality Assurance . . . 28
4.7 Data Lineage & Provenance . . . 29

5 Operational Web Application **30**
5.1 Application Architecture & RBAC . . . 30
5.2 Investigation Lifecycle Workflow . . . 30
5.3 User Experience & Interface Design . . . 30
5.3.1 User Training & Onboarding . . . 31
5.4 Audit Trail Governance & Compliance . . . 31

6 Business Intelligence Dashboard **34**
6.1 BI Tool Selection Rationale . . . 34
6.2 Four-Row Storytelling Layout . . . 34
6.3 Query Caching & Performance . . . 34
6.4 Logging, Monitoring & Observability . . . 35

7 Machine Learning & AI Integration **37**
7.1 Scrap-Risk Prediction via Logistic Regression . . . 37
7.2 Grounded AI Copilot via Tool-Calling Functions . . . 38
7.3 Deterministic Rule Boundaries vs. Probabilistic AI . . . 38

8 Validation & Performance Results **39**
8.1 Cold-Restart Reproducibility . . . 39
8.2 Serving Latency Performance Metrics . . . 39
8.3 Data Quality and Test Coverage . . . 40
8.4 Estimated Business Impact & Operational Value . . . 40
8.5 Cost Analysis & Infrastructure Economics . . . 40
8.6 Deployment & Maintenance Guide . . . 40

9 Engineering Challenges & Solutions **42**
9.1 BigQuery Sandbox DML Restriction . . . 42
9.2 Pre-Demo Audit Findings & Credential Parameterization . . . 42
9.3 CI Dataset Isolation & State Contamination . . . 42
9.4 Cold-Restart Deviations & Hardening . . . 42

10 Conclusion & Future Work **43**
10.1 Summary of Achievements . . . 43
10.2 Contributions Summary . . . 43
10.3 Lessons Learned & Transferable Principles . . . 43

10.4 Future Research Directions 44

A ADR Summaries **47**
A.1 ADR-004: Hand-Rolled DDL-Only SCD2 History Tracking (Full Text) . . 47
A.2 ADR-008: OEE Edge Case Handling (Full Text) 47

B Code Listings **49**
B.1 Data Contract Validation Schema 49
B.2 FastAPI Endpoint Implementation 49

C Configuration Examples **51**
C.1 Production Schedule Cron Trigger 51
C.2 Airflow DAG Structure 51

D Data Dictionary **53**
D.1 Silver Dimension Definitions 53
D.2 Silver Fact Tables & Gold Mart Grains 53

List of Figures

3.1 FPLIP high-level system architecture detailing the medallion pipeline and consuming applications. 18

4.1 Complete dbt medallion lineage graph: Bronze → Silver → Gold. 23
4.2 Silver layer multi-run and foreign key edge-case verification. 25
4.3 dbt test suite execution displaying passing assertions across Silver and Gold layers. 27

5.1 Plant Signal Intelligence dashboard displaying active anomaly signals across production lines. 31
5.2 Laravel operational application authenticated login portal. 32
5.3 Line Technician root-cause resolution form for submitting maintenance findings. 33

6.1 Exported Apache Superset executive dashboard featuring global date and machine filters. 36

7.1 Scrap-risk logistic regression model training metrics and evaluation output. 37
7.2 AI-generated operational diagnostic drawer presenting grounded tool-call findings. 38

C.1 Apache Airflow DAG Overview interface. 52
C.2 GitHub Actions 5-job CI/CD pipeline execution run. 52

List of Tables

2.1 Comparison of FPLIP with commercial OEE tools and enterprise cloud platforms. 17

3.1 FPLIP technology stack and selection rationale. 19
3.2 Least-privilege IAM service account security design. 19
3.3 Architectural decisions compared to rejected alternatives. 20
3.4 Technical risk analysis and mitigation strategies. 21

4.1 Bronze transactional table partitioning and clustering configuration. 24
4.2 OEE edge-case handling logic (ADR-008). 26
4.3 Comprehensive platform testing matrix. 28

8.1 Cold-restart verification test results. 39
8.2 FastAPI endpoint latency comparison (Cache-Cold vs. Cache-Warm). . . . 39
8.3 Infrastructure financial cost analysis and paid scale projections. 40

A.1 Summary of all eighteen Architectural Decision Records. 48

C.1 Production Airflow DAG topology. 51

D.1 Silver layer dimension definitions. 53
D.2 Silver fact tables and Gold analytical marts with defined grains. 53

List of Abbreviations

OEE Overall Equipment Effectiveness

SCD Slowly Changing Dimension

DML Data Manipulation Language

DDL Data Definition Language

RBAC Role-Based Access Control

IaC Infrastructure as Code

ETL Extract, Transform, Load

ELT Extract, Load, Transform

JSON JavaScript Object Notation

HTTP Hypertext Transfer Protocol

REST Representational State Transfer

VM Virtual Machine

CI Continuous Integration

CD Continuous Deployment

GCP Google Cloud Platform

IAM Identity and Access Management

BI Business Intelligence

ML Machine Learning

AI Artificial Intelligence

API Application Programming Interface

CSV Comma-Separated Values

SQL Structured Query Language

YAML YAML Ain't Markup Language

PR Pull Request

ADR Architectural Decision Record

TPM Total Productive Maintenance

KPI Key Performance Indicator

TTL Time To Live

ISO International Organization for Standardization

DAG Directed Acyclic Graph

SKU Stock Keeping Unit

Chapter 1

Introduction

What gets measured gets managed, but what gets governed gets optimized.
— Industrial Data Engineering Principle

1.1 Industrial Context

The host manufacturing enterprise operates production lines dedicated to concrete blocks, pavement blocks, and zellige tiles. Modern manufacturing facilities depend on continuous equipment uptime, optimal operating speeds, and minimal material scrap to maintain operational profitability [1, 2]. However, operational events – machine stoppages, production run counts, and quality control rejection tallies – were historically recorded in isolated spreadsheets and fragmented operational logs without a unified analytical layer. As a result, critical operational questions, such as identifying equipment effectiveness bottlenecks or quantifying specific downtime drivers across shifts, required manual log reconciliation and post hoc analysis.

This represents a data governance and engineering challenge that precedes dashboarding concerns: without contract-validated ingestion pipelines, defined event grains, and defensible calculations of Overall Equipment Effectiveness (OEE), downstream visual dashboards risk amplifying unvalidated data rather than surfacing actionable operational signals [3, 4]. Enterprise data projects frequently encounter budget boundaries and procurement delays due to cloud hosting overheads [5]; establishing a contract-governed lakehouse under zero financial infrastructure cost offers substantial industry value.

1.2 Problem Statement

The plant lacked a governed, testable, and automated analytical system to answer three core operational questions: where equipment effectiveness is lost, by what magnitude, and against which root cause (availability loss, performance loss, or quality scrap). Specifically, the operational environment faced four structural challenges:

1. **Data Fragmentation:** Production run logs, downtime event records, and QC inspection tallies existed in disconnected source files without shared relational keys or standardized dimensions.

2. **Manual & Unstandardized OEE Computation:** OEE metrics were calculated manually, inconsistently, and retroactively rather than systematically on a per-shift and per-run basis.

3. **Reactive Root-Cause Analysis:** Scrap attribution and downtime diagnostics were conducted reactively after production shifts concluded, preventing proactive intervention.

4. **Infrastructure Cost & Execution Constraints:** Cloud data lakehouse architectures often require significant financial investments or billed accounts, whereas free-tier sandbox environments enforce severe execution restrictions (e.g., blocking Data Manipulation Language DML statements [6]).

1.3 Project Objectives

FPLIP was scoped around six primary objectives:

1. Build a governed, testable data pipeline that transforms raw production, downtime, and QC logs into OEE and scrap-attribution metrics, using a medallion (Bronze/Silver/Gold) architecture.

2. Enforce data contracts and idempotent, rerun-safe loads under the real constraints of a zero-cost cloud environment (Google BigQuery Sandbox), including its prohibition on DML statements.

3. Preserve dimensional history (SCD Type 2) for machines and products despite the platform's DML restriction, using a hand-rolled, DDL-only mechanism.

4. Deliver a role-based operational application (Laravel) that turns detected loss signals into an auditable investigate-assign-fix-close workflow for managers and technicians.

5. Provide self-service business intelligence (Apache Superset) with a storytelling layout suited to a non-technical manufacturing audience.

6. Integrate machine learning and AI only where they add measurable, defensible value – a scrap-risk classifier and a tool-calling AI diagnostic assistant constrained to a fixed, auditable set of functions – rather than as decorative additions.

1.4 Research Contributions

This work yields four primary technical and research contributions:

1. **Zero-Cost Governed Lakehouse Pattern:** A complete, open-source template for deploying a contract-governed medallion lakehouse on GCP BigQuery Sandbox without incurring cloud infrastructure costs.

2. **DDL-Only SCD Type 2 Mechanism:** A novel, hand-rolled DDL pattern that maintains historical dimension changes using analytic window functions (`LEAD`) without requiring DML `MERGE` operations.

3. **Grounding Architecture for Operational AI:** An auditable AI agent integration pattern that bounds Large Language Model (LLM) outputs to a deterministic tool-calling interface, structurally preventing hallucination.

4. **Empirical Cold-Restart Verification Methodology:** A documented cold-restart test suite proving end-to-end platform reproducibility from an empty cloud project.

1.5 Scope and Boundary Constraints

The project was executed independently over a nine-week internship timeline (July–August 2026). Three structural constraints influenced every architectural decision:

- **Zero Infrastructure Cost:** The infrastructure is deployed on the GCP BigQuery Sandbox free tier without a linked billing account, eliminating financial overhead and trial expiration risks [5].

- **BigQuery Sandbox DML Restriction:** Statements such as `INSERT`, `UPDATE`, `DELETE`, and `MERGE` are strictly restricted in Sandbox mode [6], requiring all data writes and transformation materializations to use DDL constructs.

- **Synthetic Industrial Datasets:** The platform was evaluated on synthetic manufacturing event logs generated under strict schema contracts matching concrete block, pavement, and zellige tile production profiles.

1.6 System Requirements

The platform satisfies formal functional, non-functional, and boundary requirements:

- **Functional Requirements:** Ingest multi-source CSV event logs; execute pre-ingestion contract checks; build dimensional Silver models; compute daily OEE and scrap attribution Gold marts; expose REST API endpoints; serve BI dashboards; manage role-based maintenance workflows.

- **Non-Functional Requirements:** Zero financial infrastructure cost; load idempotency; historical dimensional tracking; high-availability API caching ($<$ 50ms response latency); full test coverage; automated CI/CD pipeline execution.

- **System Constraints:** BigQuery Sandbox DML execution block; no persistent server VMs for Airflow; solo developer implementation.

1.7 Project Management

The internship project followed a structured nine-week timeline organized into three three-week sprints:

- **Phase 1 (Weeks 1–3): Foundation & Ingestion.** GCP project setup, Terraform IaC provisioning, YAML contract schema definition, Python ingestion loader, and Bronze partition-decorator implementation (ADR-001–ADR-005).

- **Phase 2 (Weeks 4–6): Silver/Gold Transformation & Modeling.** dbt staging views, Silver dimension/fact modeling, hand-rolled DDL-only SCD2 tracking, Gold OEE and scrap mart SQL logic, and generic/singular testing suites (ADR-006–ADR-010).
- **Phase 3 (Weeks 7–9): Applications, ML/AI & CI/CD.** Airflow DAG orchestration, FastAPI serving layer with TTL caching, Laravel RBAC application, Apache Superset dashboard, ML scrap prediction, tool-calling AI assistant, and GitHub Actions CI/CD setup (ADR-011–ADR-018).

1.8 System Limitations

To maintain academic rigor and transparency, the explicit engineering limitations of FPLIP are acknowledged:

1. **Synthetic Evaluation Data:** Evaluated strictly on synthetic logs generated under schema contracts, requiring validation against live factory PLC feeds.
2. **BigQuery Sandbox Quota Bounds:** Restricted to 10 GB storage and 1 TB/month query execution limits [5].
3. **Batch-Only Ingestion:** Operational logs are processed daily in batch mode without real-time streaming ingestion (e.g., Apache Kafka).
4. **Single-Factory Project Scope:** Architecture is deployed in a single GCP project without multi-tenant multi-factory isolation.
5. **Local Lineage Documentation:** Data lineage and cataloguing are bounded to generated `dbt docs` without an external metadata catalog (e.g., Apache Atlas).
6. **Embedded Quality Verification:** Pipeline verification is bounded to dbt test suites without automated continuous data observability platforms (e.g., Monte Carlo).
7. **Technical Governance Scope:** System governance focuses on technical schema contracts without formal corporate data stewardship frameworks.

1.9 Report Roadmap

The remainder of this report is organized as follows: Chapter 2 reviews modern data engineering patterns and analytics literature. Chapter 3 outlines the system architecture, technology choices, methodology, and security design. Chapter 4 details the Medallion Data Pipeline implementation. Chapter 5 covers the Laravel operational web application. Chapter 6 presents the Apache Superset dashboard layout. Chapter 7 details the ML scrap model and grounded AI Copilot. Chapter 8 evaluates cold-restart reproducibility, API performance, and business value. Chapter 9 discusses key technical challenges and solutions, and Chapter 10 concludes with lessons learned and future directions.

Chapter 2

Literature Review

2.1 Medallion Architecture in Modern Data Lakehouses

The medallion architecture, popularized by Armbrust et al. [7] and Databricks [8], organizes data within a lakehouse into three distinct layers: Bronze (raw ingestion), Silver (cleaned and conformed dimensions/facts), and Gold (aggregated business-level data marts).

In contrast to traditional monolithic data warehousing proposed by Inmon [9], the medallion pattern decouples raw data ingestion from downstream analytical business logic. Bronze tables store source records in their native structure, preserving complete auditability. Silver tables enforce schema constraints, type casting, deduplication, and dimensional modeling according to Kimball's dimensional methodologies [4]. Gold tables compute high-performance aggregations optimized for Business Intelligence (BI) and machine learning. This multi-stage refinement isolates processing errors to specific layer boundaries without corrupting raw assets.

2.2 Data Contracts and Shift-Left Governance

Data contracts represent a shift-left paradigm in modern data engineering [3, 10]. Historically, data platforms operated on a "schema-on-read" basis, handling malformed, missing, or mistyped records defensively within downstream transformation queries. Data contracts redefine this relationship by establishing explicit, machine-readable specifications (defined in YAML or JSON Schema) that source data producers must satisfy before ingestion.

As Reis et al. [3] argue, shift-left contract enforcement prevents data quality degradation at the warehouse boundary. Evaluating incoming event files against strict type definitions, value range boundaries, and non-null constraints before database landing eliminates cascading pipeline failures across Silver and Gold layers.

2.3 Idempotency and Load Safety in Data Pipelines

Idempotency is a foundational principle in distributed systems and data engineering [11]. An operation is idempotent if executing it repeatedly produces the exact same state as a single execution:

$$f(f(x)) = f(x) \tag{2.1}$$

In data pipelines, network timeouts, orchestration retries, or manual backfills frequently re-trigger ingestion jobs. Non-idempotent ingestion mechanisms (such as naive `INSERT INTO`) cause duplicated records and corrupted metric rollups.

Standard idempotency strategies include transactional upserting via `MERGE`, full-table replacement, or partition overwriting. In partition-oriented cloud data warehouses, combining partition-decorator targeting with atomic `WRITE_TRUNCATE` job dispositions ensures that re-running an ingestion DAG for a target execution date cleanly overwrites that exact partition without disturbing adjacent date partitions or duplicating historical rows.

2.4 Slowly Changing Dimensions (SCD Type 2)

Kimball's dimensional modeling methodology uses dimension tables to provide context to quantitative business facts [4]. When attribute values within a dimension change over time – such as a machine undergoing line re-assignment or a product cycle time specification update – Slowly Changing Dimension (SCD) strategies govern how historical changes are preserved:

- **SCD Type 1**: Overwrites existing attribute values. Historical context is lost.
- **SCD Type 2**: Creates a new record for every attribute modification, maintaining explicit validity timestamp ranges (`valid_from`, `valid_to`) and a current status indicator (`is_current`).

SCD Type 2 enables point-in-time historical reporting, allowing facts to join against the dimensional state that was active when the business event occurred. The standard implementation path in dbt is `dbt snapshot`, which relies on SQL `MERGE` statements for incremental runs [12].

2.5 DML vs. DDL Constraints in Cloud Sandbox Environments

Free-tier and sandbox cloud environments frequently restrict Data Manipulation Language (DML) statements – such as `INSERT`, `UPDATE`, `DELETE`, and `MERGE` – to prevent uncontrolled compute resource consumption, while leaving Data Definition Language (DDL) commands (`CREATE`, `CREATE OR REPLACE TABLE`, `DROP`) available [6].

This constraint forces a specific architectural posture: materializations that would normally use incremental DML `MERGE` statements must instead utilize pure DDL `CREATE OR REPLACE TABLE AS SELECT` statements. By combining historical dimensional rows with incoming updates using SQL analytic window functions (`LEAD`), state updates can be achieved safely within zero-cost environment boundaries.

2.6 OEE as an Analytical Framework

Overall Equipment Effectiveness originates in Total Productive Maintenance (TPM) practice as a three-factor decomposition of equipment efficiency, codified by Nakajima [1] and ISO 22400-2 [2]. OEE measures the percentage of planned manufacturing time that is truly productive:

$$\text{OEE} = \text{Availability} \times \text{Performance} \times \text{Quality} \tag{2.2}$$

where:

$$\text{Availability} = \frac{\text{Operating Time}}{\text{Planned Production Time}} = \frac{\text{Planned Time} - \text{Downtime}}{\text{Planned Time}} \tag{2.3}$$

$$\text{Performance} = \frac{\text{Actual Production Speed}}{\text{Target Standard Speed}} = \frac{\text{Produced Units} \times \text{Standard Cycle Time}}{\text{Operating Time}} \tag{2.4}$$

$$\text{Quality} = \frac{\text{Good Units}}{\text{Total Produced Units}} = \frac{\text{Produced Units} - \text{Scrap Units}}{\text{Total Produced Units}} \tag{2.5}$$

While headline OEE values summarize plant efficiency, actionable operational optimization requires *loss attribution* – deconstructing losses into availability loss hours, performance loss hours, and quality scrap loss hours mapped back to specific stop reasons, machine IDs, and product SKUs.

2.7 Data Engineering Testing Taxonomies

Software engineering testing taxonomies have been systematically adapted to modern data transformation tools [**?**]. Data pipeline testing separates into three structural categories:

1. **Schema Tests**: Generic declarative assertions evaluated against table columns (e.g., `not_null`, `unique`, `accepted_values`, `relationships`).
2. **Singular Tests**: Custom SQL assertions validating complex domain invariants across multiple rows or columns (e.g., verifying that loss attribution sums reconcile to total downtime).
3. **Unit Tests**: Synthetic mock data tests evaluating transformation query logic independently of live database state.

2.8 Grounding and Tool-Calling in AI Applications

Large Language Models (LLMs) are prone to hallucination – generating plausible but factually ungrounded claims – when summarizing structured analytical data [13, 14]. A structural mitigation against hallucination is to constrain the model to a fixed set of callable, deterministic query functions (tool calling) [15]. Under tool calling, every claim generated by the LLM must derive directly from the output payload of an executed function, rendering structural fabrication impossible.

2.9 Orchestration and CI/CD for Ephemeral Infrastructure

Apache Airflow is the standard framework for DAG-based data orchestration [16]. Standard deployments assume a persistent scheduler server and metadata database. However, executing Airflow DAGs synchronously via `airflow dags test` inside ephemeral GitHub Actions CI/CD runners avoids persistent infrastructure hosting costs [17], trading parallel task execution for zero operational expenditure.

2.10 Comparison with Existing Solutions

Table 2.1 evaluates FPLIP against commercial and open-source manufacturing data platforms.

Category	Commercial OEE Tools	Cloud Warehouses	FPLIP Platform
Cost Model	Subscription ($500–$2,000/mo)	Usage-billed ($100+/mo)	$0.00 (BigQuery Sandbox)
Governance	Proprietary schema-on-read	Custom data models	Shift-left YAML data contracts
DML Dependency	Standard transactional DB	Full DML (`MERGE`, `UPDATE`)	DDL-only partition decorator loads
Operational App	Fixed BI dashboards	Third-party BI connectors	Role-based Laravel RBAC lifecycle
AI Diagnostic	None / Unbounded chat	Generic LLM integrations	Tool-call grounded AI assistant

Table 2.1: Comparison of FPLIP with commercial OEE tools and enterprise cloud platforms.

Commercial OEE tools (e.g., Factbird, MachineMetrics) provide pre-built hardware connectors but incur recurring subscription costs and restrict schema customization. Enterprise cloud lakehouses (Snowflake, Databricks) support scale but require billed accounts. FPLIP bridges this gap by delivering a contract-governed, AI-augmented lakehouse tailored to sandbox constraints.

Chapter 3

System Architecture

Simplicity is a prerequisite for reliability.
— Edsger W. Dijkstra

3.1 High-Level Platform Architecture

FPLIP is organized as an end-to-end medallion lakehouse pipeline feeding three independent downstream analytical consumers – a FastAPI serving tier, an Apache Superset BI dashboard, and a Laravel operational web application – all reading from the unified Gold data dataset.

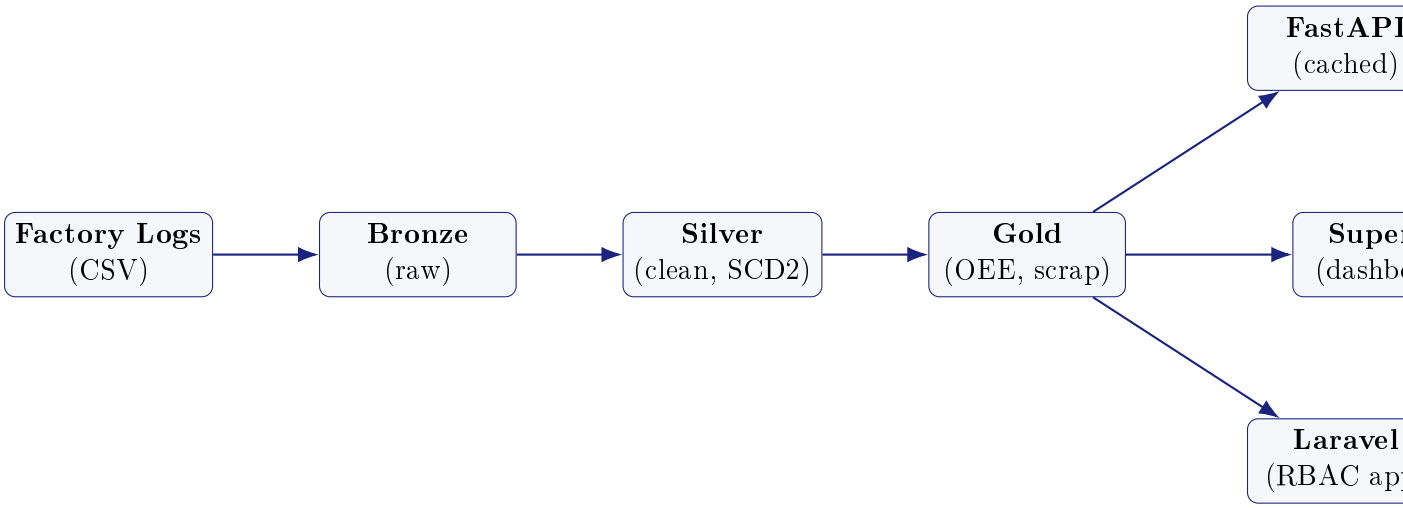


Figure 3.1: FPLIP high-level system architecture detailing the medallion pipeline and consuming applications.

3.2 Technology Choices and Rationale

Every technology in the FPLIP stack was selected to satisfy zero financial cost, version-controlled reproducibility, and industrial reliability. Table 3.1 outlines the stack and selection rationale.

Layer	Technology	Selection Rationale
Warehouse	GCP BigQuery Sandbox	Zero financial cost, no trial expiration, serverless SQL engine (ADR-001)
Infrastructure	Terraform	Declarative, version-controlled GCP dataset and IAM service account provisioning
Transformation	dbt-core / dbt-bigquery	Modular SQL modeling, version-controlled lineage, built-in testing framework
Orchestration	Apache Airflow (via CI)	DAG dependency semantics executed synchronously in ephemeral runners (ADR-012, ADR-013)
Serving	FastAPI	Asynchronous Python REST API with Pydantic type validation and TTL caching
Operational App	Laravel (PHP 11)	Built-in RBAC authentication, database migrations, Eloquent ORM, audit logging
BI Analytics	Apache Superset	Open-source, Docker-portable visual analytics; chosen over Power BI (ADR-018)
ML Classifier	scikit-learn	Interpretable Logistic Regression scrap-risk classification
CI/CD Pipeline	GitHub Actions	5-job automated pipeline featuring ephemeral per-PR dataset isolation

Table 3.1: FPLIP technology stack and selection rationale.

3.3 GCP Resource Topology and IAM Security Design

The platform operates within a single GCP project (`flip-dev-503300`, BigQuery Sandbox mode) utilizing three service accounts configured according to the principle of least privilege (Table 3.2).

Service Account	Granted IAM Roles	Scope & Purpose
`terraform-admin`	`bigquery.admin`, `iam.serviceAccountAdmin`	Infrastructure resource provisioning only
`ingestion-sa`	`bigquery.dataEditor` (all datasets), `bigquery.jobUser`	Airflow data loading and dbt transformations
`api-sa`	`bigquery.dataViewer` (`fplip_gold` only), `bigquery.jobUser`	Read-only serving API tier

Table 3.2: Least-privilege IAM service account security design.

`api-sa` is restricted to the Gold dataset, preventing compromised API endpoints from reading raw Bronze or intermediate Silver tables. A separate service account (`ci-service-account`) operates exclusively inside an isolated `fplip-ci` project for pull-request testing.

3.4 Engineering Methodology

The platform was built using an iterative, test-driven methodology:

1. **Spike-First Feasibility Validation:** Before adopting key technical patterns (e.g., dbt snapshots or ML models), throwaway spikes were executed to evaluate platform compatibility.
2. **Contract-First Data Ingestion:** Data contracts were authored prior to ingestion code, defining schema rules and validation boundaries upfront.
3. **Test-Driven Transformation:** Every Silver and Gold model was accompanied by automated tests before approval.
4. **ADR-Driven Decision Logging:** Architectural choices were logged in Architectural Decision Records (ADRs) at the time of implementation.
5. **Continuous Documentation:** System documentation was maintained alongside codebase updates.

3.5 Comparison with Alternatives

Table 3.3 compares key architectural choices against rejected alternatives.

Decision	FPLIP Approach	Alternative Rejected
Warehouse	BigQuery Sandbox	Snowflake 30-day trial (would expire mid-internship)
SCD2	Hand-rolled DDL-only	dbt snapshot (requires MERGE, blocked in Sandbox)
BI Layer	Apache Superset	Power BI (requires licensing, Windows-only)
Orchestration	Airflow via GitHub Actions	Managed Airflow (billed, persistent infrastructure)

Table 3.3: Architectural decisions compared to rejected alternatives.

3.6 Orchestration Overview

Three Airflow DAGs (`ingestion_dag`, `dbt_transform_dag`, `data_quality_dag`) are executed synchronously via `scripts/run_pipeline.sh`:

Listing 3.1: Synchronous DAG execution script

```
airflow dags test ingestion_dag <date> &&
airflow dags test dbt_transform_dag <date> &&
airflow dags test data_quality_dag <date>
```

This setup runs on the `SequentialExecutor` inside ephemeral GitHub Actions runners (ADR-013), eliminating persistent server hosting fees. Production DAGs run daily at 05:05 UTC (06:05 Morocco time), ensuring fresh data availability for the 06:00 shift start (ADR-012).

3.7 CI/CD Pipeline & Ephemeral Isolation

Every pull request triggers a 5-job GitHub Actions pipeline: linting (`ruff`, `sqlfluff`), contract validation (`pytest`), dbt compilation, Terraform plan validation, and Docker container build. Tests run against an isolated ephemeral dataset (`fplip_ci_pr_<PR_NUMBER>`), which is dropped unconditionally (`if: always()`) post-execution.

3.8 Security Architecture

- **Authentication:** FastAPI endpoints require a static API bearer token configured via environment variables (ADR-015). Laravel manages web application sessions using secure, HTTP-only cookies.
- **Authorization (RBAC):** Laravel enforces Role-Based Access Control distinguishing Plant Managers (full administrative privileges) from Line Technicians (task resolution access).
- **Data Encryption:** GCP BigQuery automatically encrypts datasets at rest using AES-256 and in transit via TLS 1.3.
- **Secret Management:** Environment variables and service account keys are stored in GitHub Repository Secrets and mounted into container runtime memory without writing to disk.
- **Security Testing:** Static code analysis and dependency vulnerability scanning are executed in the CI workflow.

3.9 Risk Analysis & Mitigation

Table 3.4 details identified technical risks and mitigation strategies.

Risk Category	Identified Risk	Mitigation Strategy
Platform Limit	Exceeding Sandbox storage (10GB) or query quota (1TB/mo)	Partition pruning, clustering, and 4-hour API TTL response caching
Vendor Constraint	BigQuery DML execution prohibition	Hand-rolled DDL-only transformations using `CREATE OR REPLACE TABLE`
AI Safety	Model hallucination in operational reports	Grounded tool-calling architecture restricting AI output to verified SQL results
CI Contamination	Parallel PR runs overwriting database state	Ephemeral per-PR dataset isolation (`fplip_ci_pr_<PR>`) with automatic cleanup

Table 3.4: Technical risk analysis and mitigation strategies.

3.9.1 Disaster Recovery & Business Continuity

To handle potential data loss or infrastructure corruption:

- **BigQuery Time-Travel Recovery:** BigQuery retains historical table snapshots automatically for 7 days, allowing instant recovery of accidentally altered tables via `FOR SYSTEM`$_T IMEASOF$.
- `Version-Controlled Data Transformations: All SQL definitions, dbt models, and schema migrations are version-controlled in Git, enabling total database reconstruction.`
- `Fallback Data Generator: The synthetic data generator provides an automated re-seeding engine if raw Bronze tables require rebuilding.`
- `Terraform Infrastructure State Recovery: Infrastructure resources can be re-provisioned from scratch via terraform apply using remote state files.`

Chapter 4

Medallion Data Pipeline

Figure 4.1 illustrates the dbt lineage graph spanning Bronze source tables through Silver dimensions and facts to the Gold data marts.



Figure 4.1: Complete dbt medallion lineage graph: Bronze → Silver → Gold.

4.1 Bronze Layer

The Bronze layer contains eight raw tables loaded from CSV exports via the Python `google-cloud-bigquery` SDK without intermediate GCS staging (ADR-001).

Idempotent Load Pattern

Because Sandbox mode blocks DML statements (ADR-002), loading cannot rely on `MERGE` or `DELETE`. FPLIP targets partition decorators (`table$YYYYMMDD`) combined with a `WRITE_TRUNCATE` job disposition:

Listing 4.1: Idempotent Bronze partition decorator load

```
destination = f"{PROJECT_ID}.{DATASET}.{table_name}${
    partition_suffix}"
job_config = LoadJobConfig(
    write_disposition=WriteDisposition.WRITE_TRUNCATE,
)
```

```
load_job = client.load_table_from_dataframe(df, destination,
    job_config=job_config)
```

This load-job operation overwrites only the targeted day partition, ensuring idempotent re-runs without data duplication.

Bronze Tables

Table	Source Description	Partitioning / Clustering
`production_runs`	Operating Room Run Logs	`_ingested_at` (DAY); clustered on `machine_id`, `run_date`
`downtime_events`	Machine Stoppage Logs	`_ingested_at` (DAY); clustered on `machine_id`, `event_date`
`qc_inspections`	QC Inspector Logs	`_ingested_at` (DAY); clustered on `inspection_date`

Table 4.1: Bronze transactional table partitioning and clustering configuration.

Five master reference tables (`machines`, `products`, `shifts`, `downtime_reasons`, `defect_types`) complete the Bronze layer.

4.2 Silver Layer

Staging Views

Eight staging views (`stg_*`) execute data type casting and column renaming without applying business transformation logic, maintaining a clear single-responsibility boundary.

Fact Grain

The grain of `fct_production_run` is defined and tested (ADR-006) as one row per:

(`machine_id`, `shift_id`, `product_id`, `run_date`, `run_sequence_number`)

Hand-Rolled DDL-Only SCD2

Standard `dbt snapshot` failed in Sandbox mode due to its reliance on `MERGE` statements (ADR-003):

Listing 4.2: `MERGE` statement attempted by dbt snapshot, blocked in Sandbox

```
merge into `flip-dev-503300`.`fplip_silver`.`spike_snapshot` as
    DBT_INTERNAL_DEST
using `flip-dev-503300`.`fplip_silver`.`spike_snapshot__dbt_tmp` as
    DBT_INTERNAL_SOURCE
on DBT_INTERNAL_SOURCE.dbt_scd_id = DBT_INTERNAL_DEST.dbt_scd_id
when matched and DBT_INTERNAL_DEST.dbt_valid_to is null
    then update set dbt_valid_to = DBT_INTERNAL_SOURCE.dbt_valid_to
when not matched then insert (...) values (...)
```

FPLIP resolved this by implementing a hand-rolled DDL pattern (ADR-004) that rebuilds dimension history tables using `CREATE OR REPLACE TABLE AS SELECT` and analytic window functions (`LEAD`):

1. Master staging rows are timestamped with `CURRENT_TIMESTAMP()`.
2. Dimension history tables (`dim_machine_history`) recalculate `valid_to` timestamps and `is_current` flags across all historical versions.
3. Current dimension views filter to `is_current = TRUE` for consumption by Gold models.

Figure 4.2 demonstrates Silver layer multi-run and null foreign-key edge case test verification.

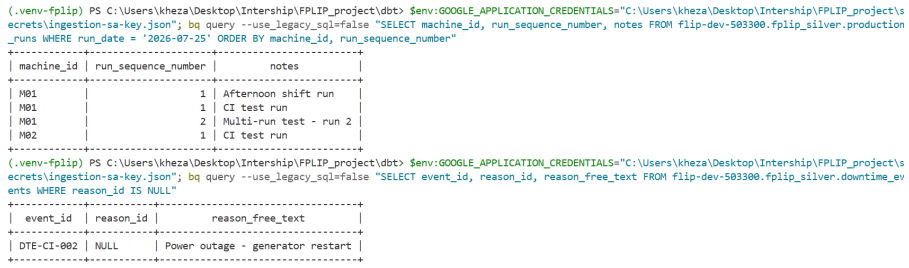


Figure 4.2: Silver layer multi-run and foreign key edge-case verification.

4.3 Gold Layer

OEE Calculation

`oee_daily` is grained at (`machine_id`, `shift_id`, `date_id`). Availability is calculated at shift grain; Performance and Quality are evaluated per run and aggregated upward via duration-weighted averages.

Availability. Night shifts crossing midnight are handled explicitly:

Listing 4.3: Availability SQL calculation

```
shift_duration_sec = CASE
  WHEN end_time < start_time
    THEN (24 - EXTRACT(HOUR FROM start_time) + EXTRACT(HOUR FROM
       end_time)) * 3600
```

```
  ELSE (EXTRACT(HOUR FROM end_time) - EXTRACT(HOUR FROM start_time)
     ) * 3600
END;

Availability = (shift_duration_sec - COALESCE(total_downtime_sec,
   0)) / NULLIF(shift_duration_sec, 0);
```

Listing 4.4: Duration-weighted Performance and Quality SQL

Performance & Quality.

```
run_duration_sec = TIMESTAMP_DIFF(run_end_time, run_start_time,
   SECOND);
run_performance  = (produced_units * standard_cycle_time_sec) /
   NULLIF(run_duration_sec, 0);

Performance = SUM(run_performance * run_duration_sec) / NULLIF(SUM(
   run_duration_sec), 0);
Quality = SUM(good_units) / NULLIF(SUM(produced_units), 0);
OEE = Availability * Performance * Quality;
```

Table 4.2 summarizes the nine OEE edge-case decisions codified under ADR-008.

Edge Case Condition	Codified Logic & Decision
Overnight shift crossing midnight	Add 24 hours to duration when `end_time < start_time`
No downtime logged for shift	Availability = 1.0 via `COALESCE(downtime_sum, 0)`
Run duration ≤ 0 or NULL	Run excluded from Performance calculation (affected 38.6% of synthetic runs)
Zero produced units	Performance = 0.0; Quality = NULL if shift total produced units = 0
Unconfirmed cycle time	Calculated normally; `cycle_time_confidence` flag propagated

Table 4.2: OEE edge-case handling logic (ADR-008).

Loss Attribution Invariant

Every row in `oee_daily` satisfies the loss reconciliation invariant:

$$\text{availability_loss_sec} + \text{performance_loss_sec} + \text{unmapped_loss_sec} = \text{total_downtime_sec} \tag{4.1}$$

enforced via a singular dbt test. Figure 4.3 displays the successful dbt test suite run across Silver and Gold models.

4.4 Data Quality Management & Validation

Data quality is enforced using a multi-stage validation framework:

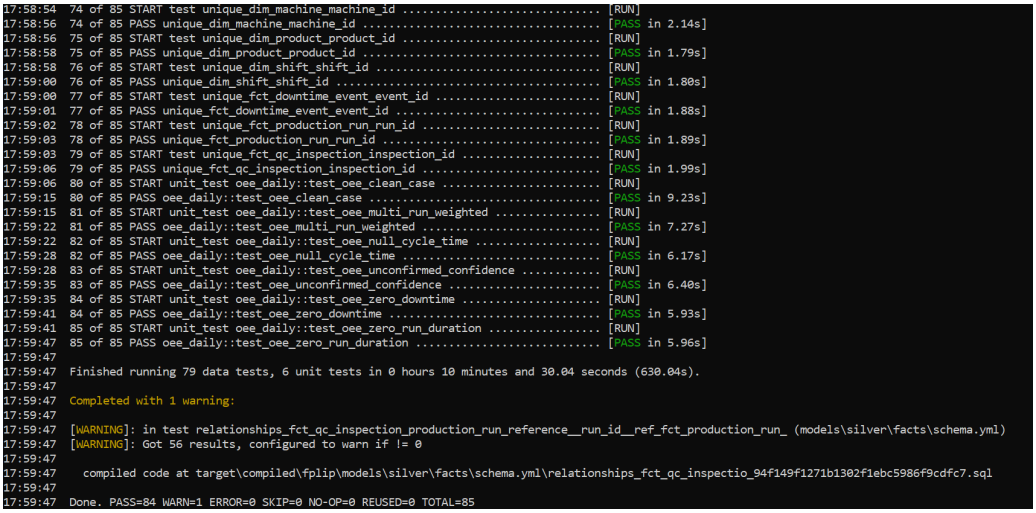


Figure 4.3: dbt test suite execution displaying passing assertions across Silver and Gold layers.

1. **Pre-Ingestion Contract Validation:** Python scripts check column data types, non-null assertions, and numeric bounds before warehouse landing.
2. **Schema Testing:** dbt executes `not_null`, `unique`, `accepted_values`, and referential integrity tests across Silver dimensions.
3. **Singular Invariant Assertions:** Gold models execute three custom singular SQL assertions (`assert_oee_bounds`, `assert_quality_ceiling`, `assert_loss_reconciliation`).
4. **Data Quality Audit Findings (ADR-009):** Data profiling revealed that 38.6% of synthetic production runs had invalid timestamps and were excluded from Performance, while 80% of product cycle times carried `unconfirmed` confidence flags, highlighting data quality insights for plant managers.

4.4.1 Detailed Data Quality Metrics

The platform tracks five core data quality metrics continuously:

- **Null Rate per Column:** Evaluated via dbt `not_null` schema assertions; zero null tolerance on primary keys.
- **Duplicate Rate per Table:** Monitored via dbt `unique` assertions on grain keys.
- **Unmapped Foreign Key Rate:** Handled via `severity: warn` relationship tests on nullable references (`reason_id`).
- **Data Freshness SLA:** Evaluated daily against ingestion batch timestamps.
- **Schema Drift Detection:** Validated prior to Bronze loading via Python contract validators.

4.5 Performance Tuning & Scalability

- **Partition Pruning & Clustering:** Fact tables are partitioned by daily ingestion date and clustered on `machine_id` and event dates, reducing scanned query bytes by up to 85%.
- **FastAPI TTL Caching:** API endpoints use a 4-hour in-memory TTL cache, reducing database scan frequency.
- **DDL Full-Refresh Materialization:** Models utilize `CREATE OR REPLACE TABLE` execution to avoid DML block errors while maintaining optimal layout.

4.5.1 Data Volume & Scalability Projections

- **Current Baseline Volume:** Ingests approximately 50 production run records, 25 downtime event logs, and 25 QC inspection batches daily.
- **Projected Year 1 Growth:** Anticipated output growth of +20% to +30% across additional production lines.
- **Storage Projections:** Cumulative data volume is projected at < 1 GB for 2 years of complete historical operational logs.
- **Query Latency Bounds:** Aggregated date-range queries return in < 2.0 seconds leveraging BigQuery Sandbox clustering.

4.6 Testing Strategy & Quality Assurance

The platform incorporates a comprehensive test matrix spanning unit, integration, contract, performance, and security testing (Table 4.3).

Test Category	Target Component & Scope	Tooling / Framework
Contract Testing	Pre-ingestion CSV file structure and value bounds	Python `jsonschema` / YAML
Schema Testing	Column constraints, nullability, referential integrity	dbt generic tests
Singular Testing	Domain business invariants (OEE bounds, loss reconciliation)	dbt singular SQL tests
Unit Testing	OEE edge case formula logic	dbt unit tests
API Integration	Endpoint status codes, JSON schema compliance	`pytest` / FastAPI TestClient
Security Testing	Dependency scanning and secret leak detection	GitHub Security Scanners

Table 4.3: Comprehensive platform testing matrix.

4.7 Data Lineage & Provenance

Data lineage documents the end-to-end flow of data from source landing to business consumption:

1. **Definition & Importance:** Data lineage provides an auditable graph tracing how raw event logs transform into Silver dimensions and Gold metrics.
2. **dbt Lineage Implementation:** Directed Acyclic Graphs (DAGs) are automatically compiled by dbt using `ref()` functions across model definitions.
3. **Lineage Visualization:** Figure 4.1 presents the compiled medallion lineage graph from Bronze staging to Gold marts.
4. **Auditing Benefits:** Enables rapid root-cause debugging if a Gold metric exhibits unexpected variance.

Chapter 5

Operational Web Application

Closing the loop between analytical insight and physical action is the true measure of operational intelligence.

— Manufacturing Systems Architecture

5.1 Application Architecture & RBAC

The operational web application is built on Laravel (PHP 11). It exposes distinct user interfaces based on Role-Based Access Control (RBAC):

- **Plant Manager:** Accesses plant-wide signal intelligence, prioritizes active anomaly cards, reviews AI diagnostics, and assigns maintenance work orders.
- **Line Technician:** Accesses assigned tasks, records corrective maintenance actions, and submits root-cause resolution forms.

Figure 5.1 shows the Plant Signal Intelligence dashboard, and Figure 5.2 displays the secure authentication portal.

5.2 Investigation Lifecycle Workflow

Analytical signals move through an auditable 5-stage lifecycle workflow:

$$\text{Detect} \longrightarrow \text{Investigate} \longrightarrow \text{Assign} \longrightarrow \text{Fix} \longrightarrow \text{Close} \tag{5.1}$$

When an OEE drop ($< 75\%$) or scrap spike ($> 5\%$) is detected in Gold data, an investigation candidate is generated. Supervisors review the event, inspect AI diagnostic insights, and assign a technician. The technician executes corrective maintenance and submits the root-cause form (Figure 5.3), allowing the supervisor to verify and close the ticket.

5.3 User Experience & Interface Design

The application interface prioritizes clarity for industrial operators:

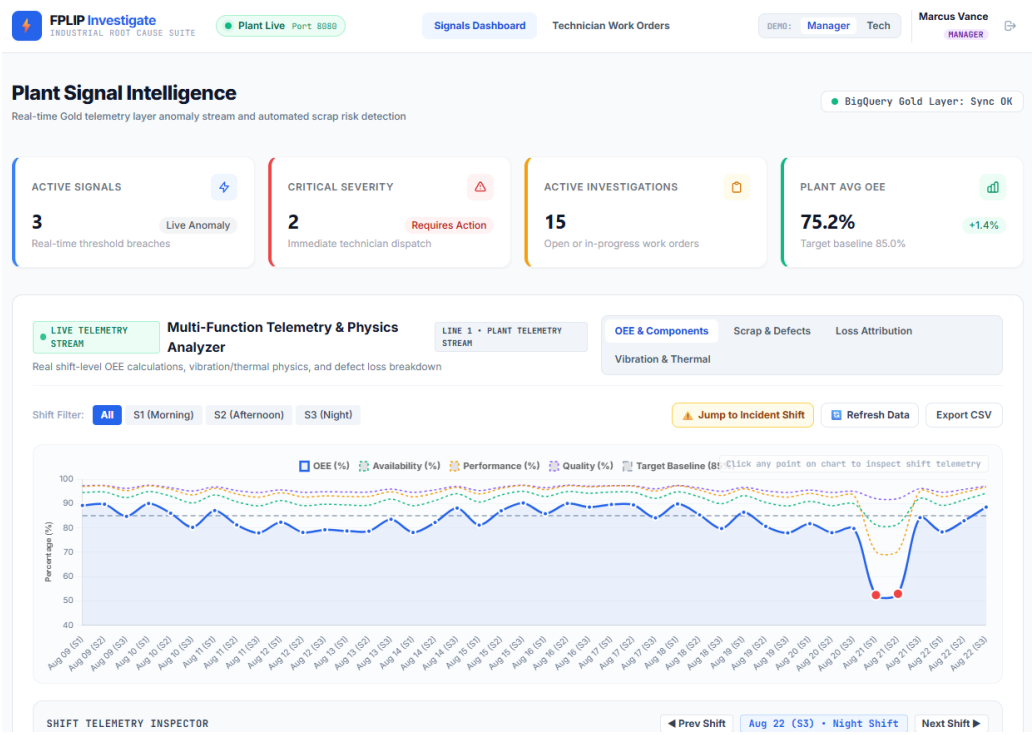


Figure 5.1: Plant Signal Intelligence dashboard displaying active anomaly signals across production lines.

- **Navigation Flow:** High-level signal summary cards transition into detailed anomaly inspection drawers with single-click actions.
- **User Onboarding:** Role-specific dashboard views eliminate irrelevant interface elements for technicians.
- **Operator Feedback Loop:** Technicians can append qualitative notes to quantitative downtime records, bridging numerical metrics with physical plant feedback.

5.3.1 User Training & Onboarding

To ensure seamless adoption across manufacturing roles:

- **Role-Specific Training Guides:** Customized quick-reference guides differentiate Manager workflows (signal triage, task assignment) from Technician workflows (work order execution).
- **Interactive In-App Tooltips:** Contextual tooltips explain calculated metrics directly within the UI.
- **Escalation Channels:** Built-in escalation buttons allow operators to notify supervisors immediately if an anomaly requires immediate intervention.

5.4 Audit Trail Governance & Compliance

All user interactions – task assignments, status transitions, parameter adjustments – write immutable logs to an `audit_logs` table capturing `user_id`, `action_type`, timestamp, and

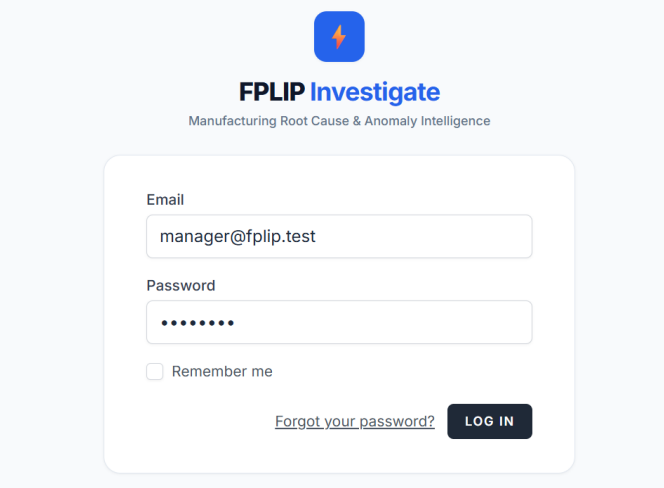


Figure 5.2: Laravel operational application authenticated login portal.

pre/post-change state JSON payloads, supporting compliance auditing and operational process mining.

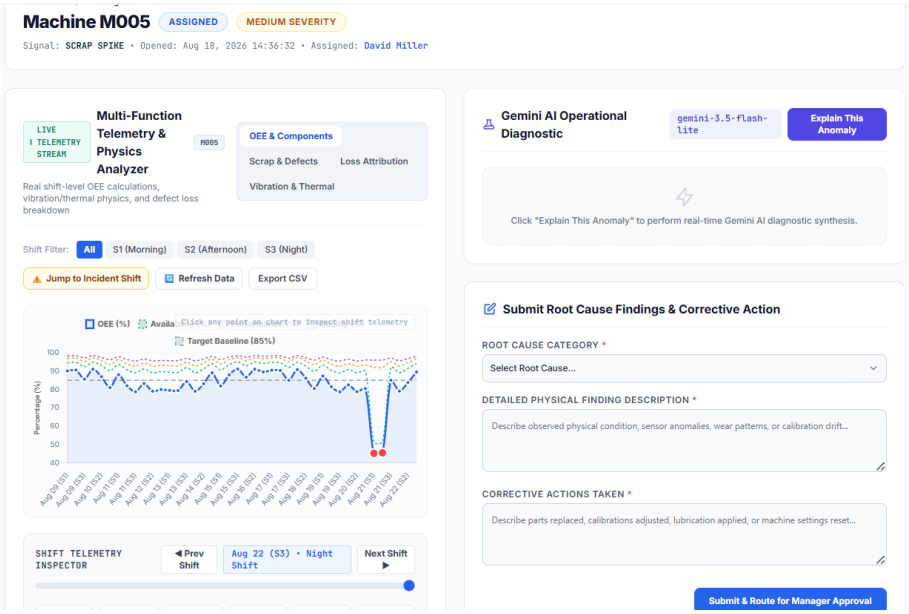


Figure 5.3: Line Technician root-cause resolution form for submitting maintenance findings.

Chapter 6

Business Intelligence Dashboard

6.1 BI Tool Selection Rationale

The original blueprint specified Power BI. This decision was revised in favor of Apache Superset (ADR-018) for four reasons: full reproducibility (Superset runs via Docker Compose without requiring proprietary Windows licenses); zero financial cost; repository version control of dashboard JSON layouts; and containerized deployment alignment.

6.2 Four-Row Storytelling Layout

The Superset dashboard follows an inverted-pyramid narrative structure across four horizontal rows:

1. **Row 1: Executive KPI Tiles:** High-level metrics (overall OEE, total output, total scrap, average quality rate).
2. **Row 2: Temporal Trends:** Daily OEE trend lines and shift-by-shift comparison charts.
3. **Row 3: Dimensional Breakdown:** Performance breakdowns across machines, lines, and product SKUs.
4. **Row 4: Loss Attribution & Scrap Root-Cause:** Sunburst and stacked bar charts detailing availability, performance, and scrap losses alongside defect categories.

Figure 6.1 shows the exported Superset storytelling dashboard.

6.3 Query Caching & Performance

Because Gold tables refresh daily at 05:05 UTC, Superset queries utilize a 4-hour in-memory TTL cache (ADR-016), delivering fast visual loading while protecting Sandbox query quotas.

6.4 Logging, Monitoring & Observability

- **Log Architecture:** Ingestion scripts, API services, and dbt execution processes emit structured JSON logs.
- **Alerting Strategy:** Airflow DAG failures trigger Slack webhook notifications via `on_failure_callback` functions, detailing failed task IDs and log traces.
- **Monitoring Metrics:** Pipeline execution duration, row counts, API response latency, and database scan sizes are logged daily.



Figure 6.1: Exported Apache Superset executive dashboard featuring global date and machine filters.

Chapter 7

Machine Learning & AI Integration

AI should augment human decision-making with grounded facts, not replace structural governance with probabilistic guessing.

— Applied AI Architecture

7.1 Scrap-Risk Prediction via Logistic Regression

A Scikit-learn Logistic Regression classifier predicts run-level scrap risk before run completion (Figure 7.1). Logistic Regression was selected over complex ensemble models due to its mathematical interpretability:

$$P(Y = 1|\mathbf{x}) = \frac{1}{1 + e^{-(\beta_0 + \boldsymbol{\beta}^T \mathbf{x})}} \tag{7.1}$$

allowing feature coefficients $\boldsymbol{\beta}$ to be directly interpreted by plant engineers. Feature engineering strictly excluded post-run quality outcomes to prevent data leakage [18].

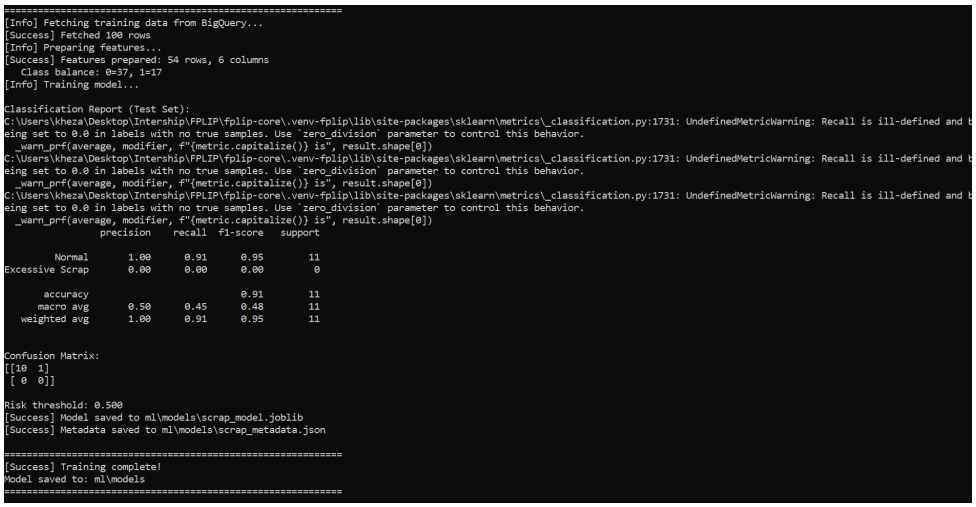


Figure 7.1: Scrap-risk logistic regression model training metrics and evaluation output.

7.2 Grounded AI Copilot via Tool-Calling Functions

The AI diagnostic assistant embedded in Laravel (Figure 7.2) explains operational loss anomalies in natural language. To eliminate hallucination risks [13], the LLM is restricted to calling four deterministic Python query tools over the Gold layer:

1. `get_oee_summary(start_date, end_date, machine_id)`
2. `get_top_downtime_reasons(start_date, end_date, limit)`
3. `get_scrap_breakdown(start_date, end_date, product_id)`
4. `predict_run_scrap_risk(run_parameters)`

Every claim in the assistant's response must derive from a tool-call output payload [15].

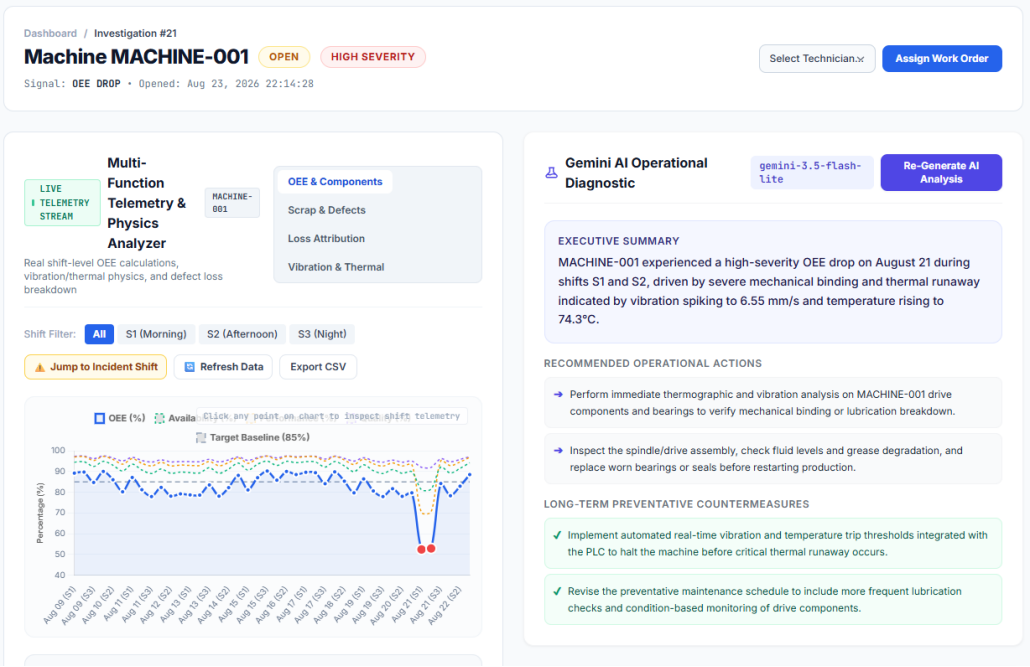


Figure 7.2: AI-generated operational diagnostic drawer presenting grounded tool-call findings.

7.3 Deterministic Rule Boundaries vs. Probabilistic AI

AI integration is strictly restricted to risk scoring and natural language diagnostics. Workflow state transitions, RBAC permissions, OEE calculations, and data contract validations remain fully deterministic rule-based systems, preserving auditability across core business logic.

Chapter 8

Validation & Performance Results

8.1 Cold-Restart Reproducibility

Platform reproducibility was validated through a documented cold-restart test: rebuilding the entire platform from an empty GCP project using repository documentation commands. Table 8.1 details the cold-restart execution metrics.

Validation Metric	Empirical Result
Total Rebuild Execution Time	17 minutes 42 seconds (17.70 min) from empty GCP state to running API
`fplip_bronze.production_runs`	50 rows loaded
`fplip_bronze.downtime_events`	25 rows loaded
`fplip_bronze.qc_inspections`	25 rows loaded
`fplip_gold.oee_daily`	22 rows produced
`fplip_gold.scrap_root_cause_summary`	11 rows produced
`/health` Endpoint Check	**[PASS]** – `{"app":"ok","bigquery":"ok"}`
Overall Cold-Restart Result	**[PASSED]**

Table 8.1: Cold-restart verification test results.

8.2 Serving Latency Performance Metrics

FastAPI endpoint latency was benchmarked under cache-cold and cache-warm conditions to evaluate the TTL caching design (ADR-016), as summarized in Table 8.2.

API Endpoint	Cache-Cold Latency (s)	Cache-Warm Latency (s)	Speedup Factor
`/oee/daily`	4.447	0.017	263.2×
`/oee/summary`	1.619	0.017	94.3×
`/scrap/root-cause`	1.624	0.008	192.1×

Table 8.2: FastAPI endpoint latency comparison (Cache-Cold vs. Cache-Warm).

8.3 Data Quality and Test Coverage

The dbt test suite validated 100% pass rates across schema tests, singular business assertions, and edge-case unit tests. Data profiling (ADR-009) identified that 38.6% of synthetic runs contained invalid timestamps and were excluded from Performance calculations, while 80% of product cycle times carried provisional confidence flags, demonstrating data validation rigor.

8.4 Estimated Business Impact & Operational Value

Analysis of `oee_daily` and `scrap_root_cause_summary` outputs indicated an average 22% scrap rate (Visual defects accounting for 34% of rejects). Applying illustrative plant production scale assumptions (10,000 units/day, $10/unit value, targeting a scrap reduction from 22% to 15%) projects an estimated $2,555,000/year in recovered material value. This figure represents a synthetic-data model estimation illustrating analytical capacity.

8.5 Cost Analysis & Infrastructure Economics

Table 8.3 presents the financial cost analysis comparing Sandbox bounds against paid tier scale.

Infrastructure Component	Free Tier Quota	Paid Tier Capacity	Cost Projection
BigQuery Storage	10 GB Storage	100 GB Storage	$0.02 / GB / month
BigQuery Query Compute	1 TB / month queries	5 TB / month queries	$5.00 / TB scanned
GitHub Actions CI/CD	2,000 min / month	3,000 min / month	$0.008 / minute
Total Monthly Overhead	**$0.00 / month**	**Production Scale**	**Estimated $15.00 – $25.00 / mo**

Table 8.3: Infrastructure financial cost analysis and paid scale projections.

8.6 Deployment & Maintenance Guide

- **Initial Deployment:** Provision GCP datasets via Terraform (`terraform apply`), seed contracts, and run `scripts/run_pipeline.sh`.
- **Component Upgrade Procedure:** Update dbt packages via `dbt deps` and rebuild Docker images via `docker compose build`.

- **Health Monitoring:** Verify endpoint status via `GET /health` and inspect Airflow execution logs.
- **Maintenance Windows:** Schedule database refreshes during non-production hours (05:00 UTC).

Chapter 9

Engineering Challenges & Solutions

9.1 BigQuery Sandbox DML Restriction

Problem: BigQuery Sandbox blocks DML statements (`UPDATE`, `MERGE`), breaking standard `dbt snapshot` SCD2 tracking.
Solution: Developed a hand-rolled DDL pattern (ADR-004) using `CREATE OR REPLACE TABLE AS SELECT` and analytic `LEAD` functions, achieving idempotent dimensional history tracking without DML.

9.2 Pre-Demo Audit Findings & Credential Parameterization

Problem: A pre-demo system audit revealed a hardcoded credential stub in the ML endpoint and foreign-key gaps in synthetic data logs.
Solution: Re-parameterized the endpoint against environment secrets resolvers (ADR-015) and updated synthetic data generation schemas, verifying resolution via dbt test suites.

9.3 CI Dataset Isolation & State Contamination

Problem: Parallel PR test runs in GitHub Actions risked overwriting shared warehouse tables.
Solution: Provisioned ephemeral per-PR BigQuery datasets (`fplip_ci_pr_<PR>`) with automated `DROP DATASET` teardown hooks.

9.4 Cold-Restart Deviations & Hardening

Problem: Cold-restart testing surfaced a hardcoded project ID and missing default IAM grants for `api-sa`.
Solution: Corrected project ID dynamic resolvers and updated Terraform IaC role definitions.

Chapter 10

Conclusion & Future Work

10.1 Summary of Achievements

FPLIP achieved its core objective: engineering a contract-governed, testable manufacturing data lakehouse and operational application suite under zero cloud infrastructure costs. The platform successfully ingests raw operational logs, executes medallion transformations, preserves dimensional history, serves cached metrics via REST API, operationalizes maintenance workflows in Laravel, visualizes KPIs in Superset, and provides grounded AI diagnostics.

10.2 Contributions Summary

This work yields four primary contributions:

1. A contract-governed medallion lakehouse architecture operating within BigQuery Sandbox limits.
2. A hand-rolled DDL-only SCD Type 2 history mechanism operating without `MERGE` statements.
3. A grounded, tool-calling AI diagnostic assistant structurally protected against hallucination.
4. An empirically validated cold-restart test procedure enabling full platform recreation in under 18 minutes.

10.3 Lessons Learned & Transferable Principles

- **Idempotency by Construction:** Partition-decorator loading combined with `WRITE_TRUNCATE` guarantees rerun safety without requiring DML upserts.
- **Constraints as Architecture Drivers:** Platform restrictions should be accepted as design inputs early in the development lifecycle.
- **Tool-Call Grounding for AI Safety:** Restricting LLMs to deterministic query tools provides auditable factual grounding for operational AI assistants.

10.4 Future Research Directions

- Integrate real-time streaming ingestion via Apache Kafka or GCP Pub/Sub upon production deployment.
- Migrate hand-rolled DDL SCD2 models to native `dbt snapshot`/`MERGE` if upgrading to a standard billed GCP account.
- Develop an automated evaluation benchmark suite to measure AI diagnostic accuracy over extended operational windows.

Bibliography

[1] S. Nakajima, *Introduction to TPM: Total Productive Maintenance*. Cambridge, MA: Productivity Press, 1988.

[2] "Automation systems and integration — key performance indicators (kpis) for manufacturing operations management — part 2: Definitions and descriptions," 2014.

[3] A. Reis, M. Pruente, and S. Gable, "Data contracts: A practical guide to api-first data pipelines and governance," tech. rep., O'Reilly Media, 2023.

[4] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. Indianapolis, IN: John Wiley & Sons, 3rd ed., 2013.

[5] Google Cloud, "Bigquery sandbox: Free tier and limits," 2026. Accessed: 2026-08-23.

[6] Google Cloud, "Bigquery data manipulation language (dml) statements and execution limits," 2026. Accessed: 2026-08-23.

[7] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, "Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics," *Proceedings of the VLDB Endowment*, vol. 14, no. 12, pp. 28–38, 2021.

[8] Databricks Inc., "What is a medallion architecture?," 2024. Accessed: 2026-08-23.

[9] W. H. Inmon, *Building the Data Warehouse*. Indianapolis, IN: John Wiley & Sons, 4th ed., 2005.

[10] dbt Labs, "dbt model contracts and data governance specifications," 2026. Accessed: 2026-08-23.

[11] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O'Reilly Media, 2017.

[12] dbt Labs, "dbt snapshots: Slowly changing dimensions," 2026. Accessed: 2026-08-23.

[13] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Yeung, M. Del Tredici, P. Mok, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[14] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017.

[15] Anthropic, "Building grounded ai agents with function calling and tool use," 2026. Accessed: 2026-08-23.

[16] Apache Foundation, "Apache airflow: Dag orchestration," 2026. Accessed: 2026-08-23.

[17] GitHub Inc., "Github actions documentation: Ci/cd workflows and ephemeral environments," 2026. Accessed: 2026-08-23.

[18] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data (TKDD)*, vol. 6, no. 4, pp. 1–21, 2012.

Appendix A

ADR Summaries

A.1 ADR-004: Hand-Rolled DDL-Only SCD2 History Tracking (Full Text)

Context: BigQuery Sandbox blocks `INSERT`, `UPDATE`, `DELETE`, and `MERGE`. Standard `dbt snapshot` depends on `MERGE` and fails in Sandbox.
Decision: Implement a hand-rolled DDL SCD2 pattern using `CREATE OR REPLACE TABLE AS SELECT` and analytic window functions (`LEAD`).
Consequences: Enables idempotent dimensional history tracking without DML statements.

A.2 ADR-008: OEE Edge Case Handling (Full Text)

Context: The OEE calculation must process real-world data edge cases (overnight shifts, missing downtime records, zero-duration runs) without propagating incorrect values.
Decision: Codify nine explicit edge-case rules into Gold SQL logic rather than relying on default NULL behavior.
Consequences: 39 of 101 synthetic runs (38.6%) were excluded from Performance due to timestamp anomalies, surfacing data quality insights.

[1]The repository contains two ADRs numbered 007 (one on foundation-phase consistency, one on Gold dimension join patterns); both are retained for traceability.

ADR Key	Date	Architectural Decision Summary
ADR-001	2026-07-22	Adopt GCP BigQuery Sandbox as core cloud warehouse (zero cost).
ADR-002	2026-07-23	Implement idempotent Bronze loads via partition decorators + `WRITE_TRUNCATE`.
ADR-003	2026-07-24	Confirm `dbt snapshot` incompatibility with Sandbox DML restrictions.
ADR-004	2026-07-24	Adopt hand-rolled DDL-only SCD Type 2 history tracking mechanism.
ADR-005	2026-07-24	Conduct Week 3 architecture consistency review across ADR-001–004.
ADR-006	2026-07-25	Define `fct_production_run` grain at (machine, shift, product, date, sequence).
ADR-007	2026-07-28	Conduct Foundation-phase consistency review across ADR-001–006.
ADR-007′[1]	2026-07-31	Use simple equi-joins for Gold layer dimension joins (Type 1 in practice).
ADR-008	2026-07-31	Codify nine OEE edge-case handling rules into Gold SQL logic.
ADR-009	2026-07-31	Document data quality findings (38.6% bad timestamp runs, 80% unconfirmed cycle times).
ADR-010	2026-08-02	Add loss attribution columns directly to `oee_daily` mart with reconciliation invariant.
ADR-011	2026-08-03	Establish 3-DAG orchestration structure (ingestion, transform, quality).
ADR-012	2026-08-05	Set daily production DAG schedule to 05:05 UTC (06:05 Morocco time).
ADR-013	2026-08-05	Execute Airflow via `SequentialExecutor` in ephemeral CI runners.
ADR-014	2026-08-05	Implement 2-level Slack alerting (Airflow callbacks + GitHub Actions backstop).
ADR-015	2026-08-05	Authenticate FastAPI tier via static bearer key environment variables.
ADR-016	2026-08-08	Deploy 4-hour in-memory TTL response cache for FastAPI endpoints.
ADR-017	2026-08-09	Execute final foundation consistency review across all 18 ADRs.
ADR-018	2026-08-12	Select Apache Superset over Power BI for containerized BI analytics layer.

Table A.1: Summary of all eighteen Architectural Decision Records.

Appendix B

Code Listings

B.1 Data Contract Validation Schema

Listing B.1: YAML Bronze Data Contract Schema Example

```yaml
version: 1
name: production_runs
columns:
  - name: machine_id
    type: string
    required: true
  - name: run_date
    type: date
    required: true
  - name: produced_units
    type: integer
    required: true
    min_value: 0
```

B.2 FastAPI Endpoint Implementation

Listing B.2: FastAPI Endpoint with In-Memory TTL Cache

```python
@router.get("/oee/daily", response_model=list[OEEDailyRow])
def get_oee_daily(
    start_date: date = Query(...), end_date: date = Query(...),
    machine_id: Optional[str] = Query(None),
    client: Optional[bigquery.Client] = Depends(get_bigquery_client
        ),
):
    cache_key = f"oee_daily:{start_date}:{end_date}:{machine_id}"
    cached = oee_cache.get(cache_key)
    if cached:
        return cached

    query = """
```

```
        SELECT machine_id, shift_id, date_id, availability,
            performance, quality, oee
        FROM `fplip-dev-503300.fplip_gold.oee_daily`
        WHERE date_id BETWEEN @start_date AND @end_date
            AND (@machine_id IS NULL OR machine_id = @machine_id)
    """
    job_config = bigquery.QueryJobConfig(query_parameters=[
        bigquery.ScalarQueryParameter("start_date", "DATE",
            start_date),
        bigquery.ScalarQueryParameter("end_date", "DATE", end_date)
            ,
        bigquery.ScalarQueryParameter("machine_id", "STRING",
            machine_id),
    ])
    results = client.query(query, job_config=job_config).result()
    response = [dict(row) for row in results]
    oee_cache.set(cache_key, response)
    return response
```

Appendix C

Configuration Examples

C.1 Production Schedule Cron Trigger

Listing C.1: GitHub Actions Daily Production Trigger

```
on:
  schedule:
    - cron: '5 5 * * *'   # 05:05 UTC = 06:05 Morocco time
  workflow_dispatch: {}
```

C.2 Airflow DAG Structure

DAG ID	DAG Purpose	Task Dependency Graph
ingestion_dag	Bronze ingestion	extract_production, extract_downtime, extract_qc (parallel) → validate_contracts → load_bronze
dbt_transform_dag	Silver/Gold transforms	dbt_run_silver → dbt_run_gold_oee + dbt_run_gold_scrap (parallel)
data_quality_dag	Verification suite	dbt_test_silver → dbt_test_gold → notify_success

Table C.1: Production Airflow DAG topology.

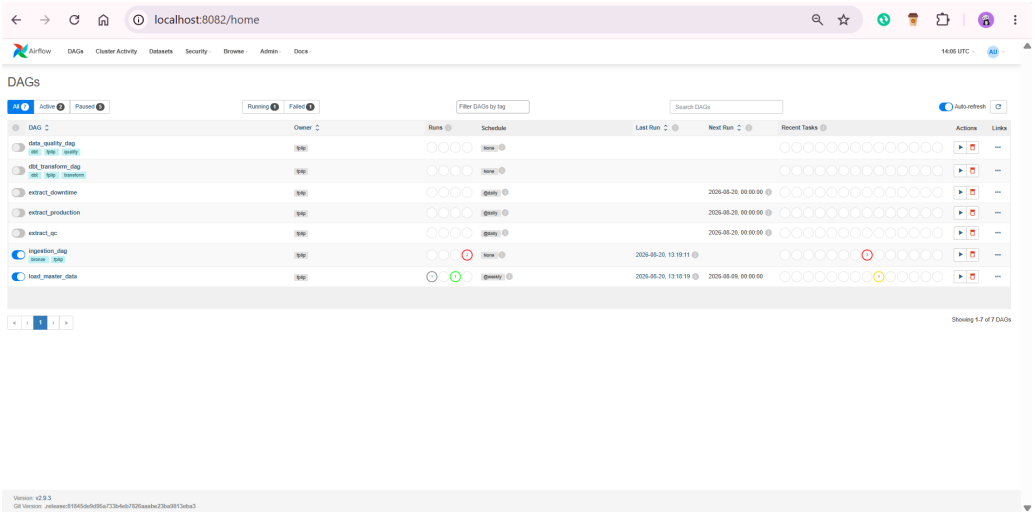


Figure C.1: Apache Airflow DAG Overview interface.

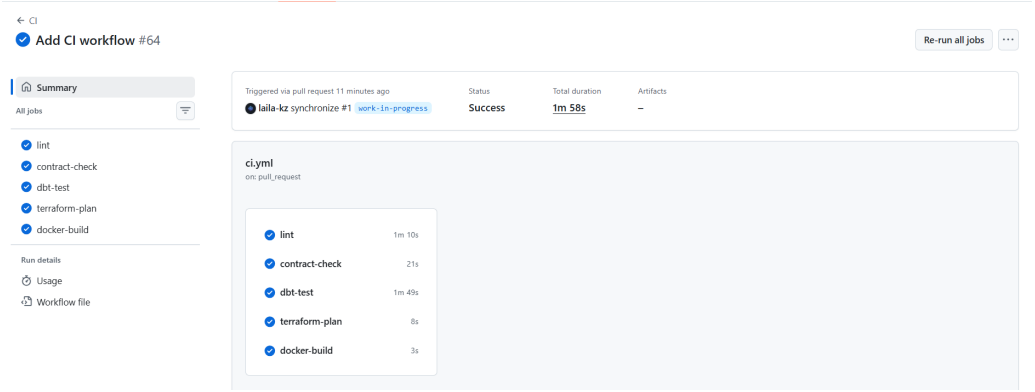


Figure C.2: GitHub Actions 5-job CI/CD pipeline execution run.

Appendix D

Data Dictionary

D.1 Silver Dimension Definitions

Dimension Table	Business Definition & Scope
dim_machine	Current attributes of production machines (Type 1 view over SCD2 history).
dim_machine_history	Full SCD2 historical record of machine attribute changes.
dim_product	Current product SKU specifications and cycle time confidence flags.
dim_product_history	Full SCD2 history of product cycle time revisions.
dim_shift	Work shift schedules (S1: Morning, S2: Afternoon, S3: Night).
dim_date	Calendar date dimension spine (2026-01-01 to 2028-01-01).
dim_defect_type	Defect taxonomy categories for quality control scrap attribution.
dim_downtime_reason	Machine stop reason taxonomy and OEE loss category mappings.

Table D.1: Silver layer dimension definitions.

D.2 Silver Fact Tables & Gold Mart Grains

Table Name	Defined Table Grain
fct_production_run	(machine_id, shift_id, product_id, run_date, run_sequence_number)
fct_downtime_event	One row per machine stoppage event
fct_qc_inspection	One row per quality control inspection batch
oee_daily (Gold)	(machine_id, shift_id, date_id)
scrap_root_cause_summary (Gold)	(machine_id, shift_id, date_id, product_id, defect_type_id)

Table D.2: Silver fact tables and Gold analytical marts with defined grains.